

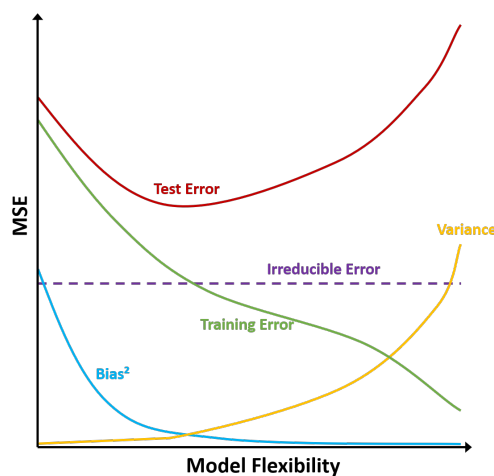
Homework problems:

Exercise 8.1. We want to take a closer look at the bias-variance trade-off and understand how it changes as the model's flexibility (i.e., the range of possible shapes for estimating the function f_0) increases.

- Create a sketch that illustrates typical curves for squared bias, variance, training error (training MSE), test error (test MSE), and irreducible error, as statistical learning methods progress from less flexible (such as simple linear regression) to more flexible approaches (such as non-parametric methods). The x-axis should indicate the degree of flexibility in the method, while the y-axis should display the corresponding values for each curve. Include all five curves and ensure each is clearly labeled.
- Explain the rationale behind the specific shapes of each of the five curves presented in part a).

Solution:

- The provided curves illustrate general trends in how the respective values change as model flexibility varies. Although the precise curves may differ for specific model classes, the overall shape remains consistent.



b)

- Bias:* As model flexibility increases, bias decreases since the model can more accurately represent the true underlying patterns in the data.
- Variance:* The variance of an estimator increases as model flexibility grows, since highly flexible models may fit the noise and random fluctuations in the training data. This results in greater variability in predictions across different training datasets.
- Training Error:* Training mean squared error (MSE) generally decreases as model flexibility increases, since more flexible models can better capture the complexities and patterns present in the training data.
- Test Error:* Test MSE often increases with greater model flexibility due to overfitting. Although flexible models may achieve low training MSE, they tend to generalize poorly to unseen data, resulting in higher test MSE.

- *Irreducible error*: The variance of the irreducible error remains unaffected by changes in model flexibility, as it reflects inherent noise or randomness in the data that no model can reduce. The irreducible error establishes a baseline for test MSE that cannot be eliminated.

Exercise 8.2. This exercise uses the **College** dataset, available in the file **College.csv** on Moodle. The dataset contains several variables for 777 universities and colleges in the United States. The variables are as follows:

- **Name**: Name of the university/college
- **Private**: Public/private indicator
- **Apps**: Number of applications received
- **Accept**: Number of applicants accepted
- **Enroll**: Number of new students enrolled
- **Top10perc**: Percentage of students from top 10 % of high school class
- **Top25perc**: Percentage of students from top 25 % of high school class
- **F.Undergrad**: Number of full-time undergraduates
- **P.Undergrad**: Number of part-time undergraduates
- **Outstate**: Tuition fees for non-citizens
- **Room.Board**: Room and board costs
- **Books**: Estimated book costs
- **Personal**: Estimated personal spending
- **PhD**: Percent of faculty with Ph.D.'s
- **Terminal**: Percent of faculty with terminal degree
- **S.F.Ratio**: Student/faculty ratio
- **perc.alumni**: Percent of alumni who donate
- **Expend**: Instructional expenditure per student
- **Grad.Rate**: Graduation rate

- a) A starter file named **starter-college.qmd** is provided on Moodle. Save both the starter file and the CSV file so that the data can be read using the following command:

```
College <- read_csv("data/College.csv")
```

- b) Summarize the dataset by calculating the median value for each numeric variable. Consult the help page for the **across()** function from **dplyr** to perform this task efficiently.

Hint: You want to summarize **across()** all variables **where()** the type is **is.numeric()**.

- c) Visualize the distribution of **Outstate** for each level of **Private**.
- d) Install the **class** package. Use the **knn** function from this package to perform k-nearest neighbors classification, aiming to predict the **Private** status. Begin by splitting the data into training and test sets using the following command:

```
set.seed(1)
indices <- sample(1:777, 544)
train <- College[indices,]
test <- College[-indices,]
```

Because the variables have different units, scale all variables so that they have mean zero and standard deviation one:

```
train_scale <- scale(train[, 3:19])
test_scale <- scale(test[, 3:19])
```

Proceed to use the `knn()` function. Refer to the help page to determine its application. Apply `knn()` with k values of 3 and 5. The output will provide predictions of `Private` for all test cases. For each value of k , calculate the accuracy as the proportion of correct predictions. Summarize your findings regarding the effect of the choice of k .

- e) Create a new qualitative variable named `Elite` by binning the `Top10perc` variable. Classify universities and colleges into two groups based on whether the proportion of students coming from the top 10% of their high school classes exceeds 50%. Compute a contingency table for `Elite` and `Private`. Visualize the distribution of `Outstate` for each combination of `Elite` and `Private`.

Solution:

- a) The initial step involves loading the tidyverse package.

```
library(tidyverse)
```

The data can then be imported.

```
College <- read_csv("data/College.csv")
```

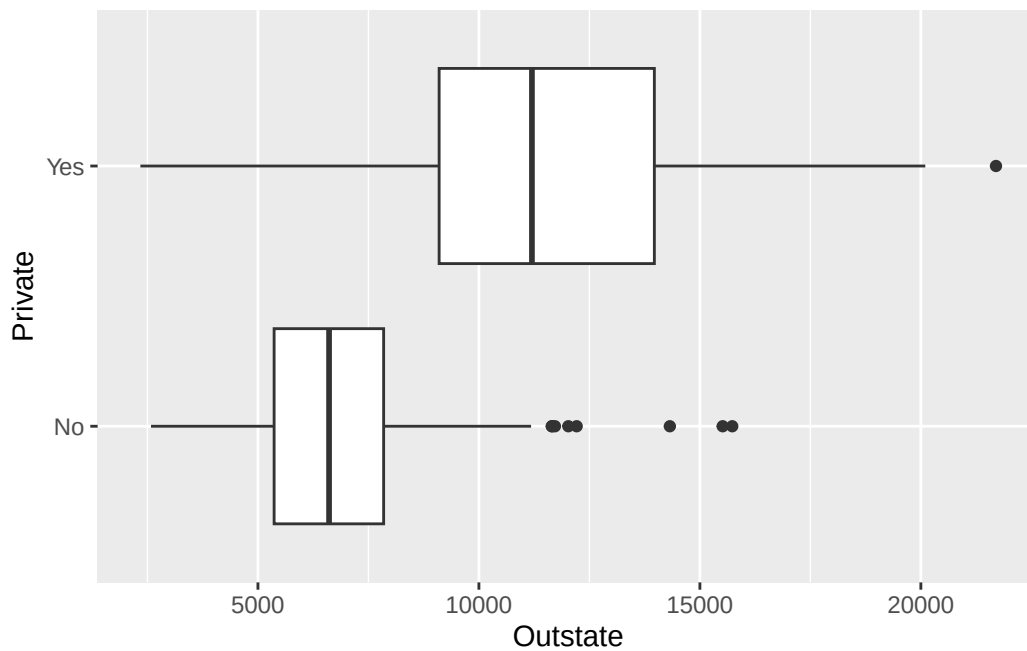
- b) The objective is to use the `across()` function to compute the median for all numeric variables. This can be accomplished with the following command.

```
College |>
  summarize(
    across(where(is.numeric), median)) |>
  print(width = Inf) # needed to show all columns
```

```
# A tibble: 1 x 17
  Apps Accept Enroll Top10perc Top25perc F.Undergrad P.Undergrad Outstate
<dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 1558 1110 434 23 54 1707 353 9990
  Room.Board Books Personal PhD Terminal S.F.Ratio perc.alumni Expend
    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 4200 500 1200 75 82 13.6 21 8377
  Grad.Rate
    <dbl>
1 65
```

- c) `Outstate` is a numeric variable. To visualize its distribution, a histogram or boxplot may be used. Comparing distributions across the two levels of `Private` is more straightforward with a boxplot:

```
ggplot(College, aes(x = Outstate, y = Private)) +
  geom_boxplot()
```



- d) We begin by creating training and test datasets. As specified in the problem, the data are standardized to ensure all variables contribute equally when calculating distances.

```
# Splitting data into train and test data
# training data should be roughly 70%: 777*0.7 around 544
set.seed(1)
indices <- sample(1:777, 544)
train <- College[indices,]
test <- College[-indices,]

# Scale all predictors
train_scale <- scale(train[, 3:19])
test_scale <- scale(test[, 3:19])
```

The k-nearest neighbors algorithm is then applied, beginning with k=3 neighbors.

```
library(class)

classifier_knn_3 <- knn(train = train_scale,
  test = test_scale,
  cl = train$Private,
  k = 3)
```

The algorithm is subsequently re-run with k=5.

```
classifier_knn_5 <- knn(train = train_scale,
  test = test_scale,
  cl = train$Private,
  k = 5)
```

Accuracy is determined by comparing observed test values with the algorithm's predictions. The proportion of correct predictions serves as the measure of accuracy.

```
mean(classifier_knn_3 == test$Private)
```

```
[1] 0.9313305
```

```
mean(classifier_knn_5 == test$Private)
```

```
[1] 0.9356223
```

A slight increase in accuracy is observed when using 5 neighbors instead of 3 for the selected test data. Examination of the contingency tables

```
table(classifier_knn_3, test$Private)
```

classifier_knn_3	No	Yes
No	62	7
Yes	9	155

```
table(classifier_knn_5, test$Private)
```

classifier_knn_5	No	Yes
No	60	4
Yes	11	158

indicates that correctly classifying public universities and colleges is more challenging than classifying private institutions.

- e) An institution is classified as **Elite** if it has a **Top10perc** value greater than 50. This variable can be created using `mutate()` in combination with `case_when()`.

```
College <- College |>
  mutate(Elite = case_when(
    Top10perc > 50 ~ "Yes",
    Top10perc <= 50 ~ "No"
  ))
```

The contingency table is computed using the following command.

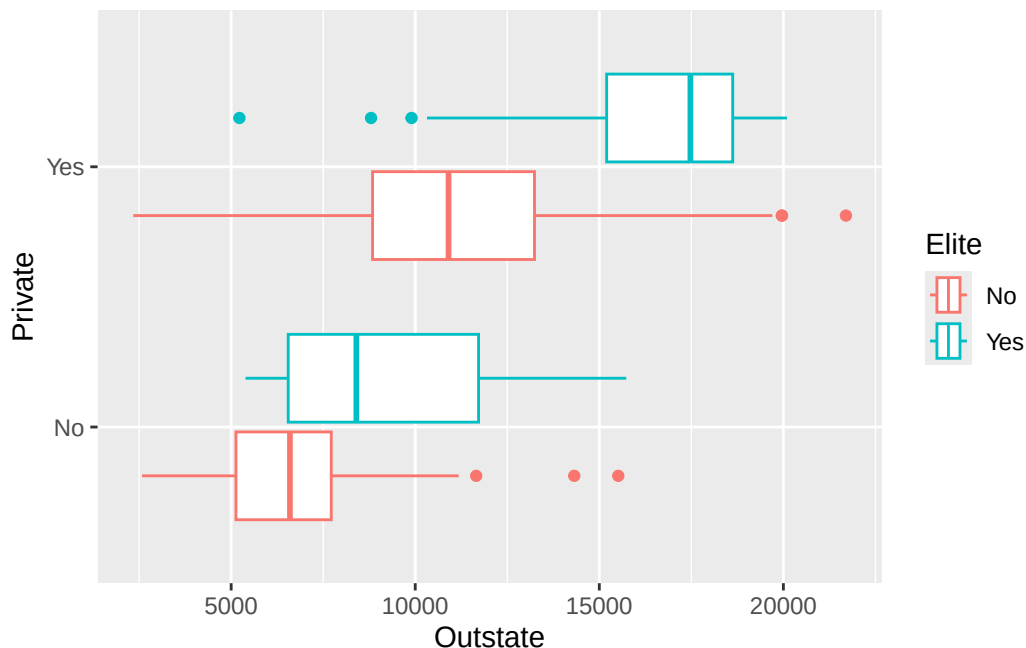
```
College |>
  select(Elite, Private) |>
  table()
```

	Private	
Elite	No	Yes
No	199	500
Yes	13	65

The table demonstrates that the percentage of **Elite** institutions is nearly twice as high for private universities compared to public ones (6 percent versus almost 12 percent). However, **Elite** status is not exclusive to private universities.

The distribution of **Outstate** for each combination of **Private** and **Elite** should be visualized. The boxplot from part c) can be used, with color indicating **Elite** categories. This approach results in two boxplots for each level of **Private**.

```
ggplot(College, aes(x = Outstate, y = Private, colour = Elite)) +  
  geom_boxplot()
```



The results indicate that, on average, the highest fees are paid at private universities classified as **Elite** institutions according to the defined criteria.

Exercise 8.3. A dataset comprising 100 observations includes a single predictor X and a quantitative response Y . A linear regression model is fitted to a training set of 80 randomly selected observations, while a separate cubic regression model, defined as $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \epsilon$, is also fitted. Both models are fitted using least squares estimation.

- Suppose the true relationship between X and Y is linear, specifically $Y = \beta_0 + \beta_1 X + \epsilon$. Consider the training residual sum of squares (RSS) for both the linear and cubic regression models. Would one model be expected to have a lower training RSS, would they be the same, or is there insufficient information to determine this? Provide justification for your answer.
- Assume the test RSS is used to compare the two fitted models instead of the training RSS. Would one model be expected to have a lower test RSS, would they be the same, or is there insufficient information to determine this? Provide justification for your answer.
- Now suppose the true relationship between X and Y is not linear, although the degree of nonlinearity is unknown. Consider the training RSS for both the linear and cubic regression models. Would one model be expected to have a lower training RSS, would they be the same, or is there insufficient information to determine this? Provide justification for your answer.
- Now assume the test RSS is used to compare the two fitted models, under the assumption that the true relationship between X and Y is not linear. Would one model be expected to have a lower test RSS, would they be the same, or is there insufficient information to determine this? Provide justification for your answer.

Solution:

- The cubic regression model offers greater flexibility compared to the linear regression model. The linear model is a special case of the cubic model when $\beta_2 = 0$ and $\beta_3 = 0$.

When fitting models to training data, a more flexible model will always achieve a training RSS that is no greater than that of a less flexible model, provided the less flexible model is nested within the more flexible one.

In this scenario, the cubic model contains more parameters ($p = 4$) than the linear model ($p = 2$). The inclusion of additional polynomial terms, specifically X^2 and X^3 , provides the cubic regression model with greater degrees of freedom to fit the training data. Even if the true relationship is linear, the cubic model will attempt to minimize the sum of squared errors by assigning small, non-zero coefficients to X^2 and X^3 , potentially reducing the training RSS further, despite these terms not being part of the true data-generating process.

Therefore, the training RSS for the cubic regression model is expected to be lower than that of the linear regression model.

- b) The test RSS is calculated using a set of observations not included in the training data, serving as a measure of the model's generalization to unseen data. Given that the true relationship between X and Y is linear, the linear regression model represents the correct specification. In contrast, the cubic regression model, despite its flexibility, incorporates unnecessary terms (X^2 and X^3) that are absent from the true data-generating process.

When a model is excessively flexible and includes irrelevant predictors, such as higher-order polynomial terms that do not represent the true relationship, it tends to overfit the training data. Overfitting occurs when the model captures both the underlying signal and the random noise present in the training data. Although this results in a lower training RSS, it typically leads to a higher test RSS.

The linear model, being the correct specification, is expected to capture the true underlying linear relationship efficiently without trying to fit the noise with unnecessary complexity. Therefore, we would expect the test RSS for the linear regression to be lower than the test RSS for the cubic regression.

- c) Consider the training RSS for both models when the true relationship is not linear. The cubic regression model is more flexible than the linear regression model, and the linear model is nested within the cubic model. Regardless of the true underlying relationship, whether linear, quadratic, cubic, or otherwise, when comparing the training RSS of two models where one is a more flexible version of the other, the more flexible model will always achieve a training RSS that is less than or equal to that of the less flexible model. This occurs because the optimization algorithm, such as least squares, for the more flexible model operates within a larger solution space. It can always identify coefficients that are as effective as, or superior to, those found by the less flexible model in minimizing the training RSS. Therefore, the training RSS for the cubic regression model is expected to be lower than that of the linear regression model.
- d) When using test RSS to compare the two fitted models, and assuming the true relationship between X and Y is not linear, the situation is more nuanced than simply selecting the more flexible model. In this context, the bias-variance trade-off is critical. The bias-variance trade-off for the two models is discussed below:

- *Linear Regression*: Bias: Since the true relationship is not linear, the linear model is likely to exhibit high bias. This model is too simple to capture the true non-linear pattern, resulting in systematic errors in its predictions and underfitting the data. Variance: Because it is a simple model, the linear regression tends to have low variance. Its predictions are relatively stable across different training sets.
- *Cubic Regression*: Bias: Since the true relationship is non-linear, the cubic model can capture more of the underlying non-linearity than the linear model. As a result, its bias is likely to be lower, particularly if the true relationship is well-approximated by a cubic polynomial or a similar function. Variance: As a more flexible model, the cubic regression includes additional parameters. This increased flexibility makes it more sensitive to noise in the training data, resulting in higher variance. The model may capture noise as if it

were a true signal, which can reduce its performance on unseen test data. Overfitting may occur if the model's complexity is excessive relative to the true relationship and the size of the dataset.

Conclusion: There is insufficient information to determine definitively which model will yield a lower test RSS.